Rank-N and Impredicative Code Synthesis in Djex and Leant

Djex and Leant contributors

September 2026

Abstract

This document describes the shared higher-rank synthesis architecture, the structural instantiation and unification improvements, the Haskell and Lean checking boundaries, and the acceptance corpus derived from `docs/examples/Church.hs`. The corpus contains 346 top-level signatures and four local signatures, including 51 partial source signatures whose holes are resolved by GHC. Every case is tested without supplying a reference implementation. There are 315 cases with no named value providers, 16 cases requiring a concrete integer constant, and 19 inherently partial cases tested under an explicit element-default assumption. Both Djinn and Exference produce 350 candidates, and GHC checks all 700 generated implementations at both their expanded query types and their original alias-preserving acceptance types. The 19 partial Haskell cases per engine additionally receive an explicitly partial wrapper at the exact original signature; bottom is inserted only at the documented default argument after synthesis. Independent live Lean runs on one unchanged executable also produce all 350 universe-correct counterparts per engine. All 700 exact displayed terms pass standalone kernel replay with empty axiom inventories. Lean retains the explicit default premise for the 19 partial source cases. These results establish a broad, reproducible practical acceptance boundary; they do not assert a terminating decision procedure for arbitrary System F inhabitation or semantic equivalence with the reference functions.

Contents

1 The problem and the supported contract

A synthesis request consists of a type T, a context Γ of available values, and operational search limits. A successful result is a term e such that

$$\Gamma \vdash e : T.$$

The names of types and functions may suggest intended behavior, but the type alone is the specification for this task. For example, an empty Church list is a valid result for a type ending in `List a`. An implementation of `sort` that always returns an empty list satisfies that type, although it does not implement sorting. Behavioral specifications, preservation of list length, or equivalence to an existing function require additional constraints and separate tests.

The implementation retains three distinct notions of evidence:

1. *Search evidence*: a backend produces a candidate through its checked representation and records any remaining constraints.
2. *Target-language evidence*: GHC or the Lean elaborator and kernel accept the emitted implementation at the requested target type.
3. *Negative evidence*: the historical complete propositional fragment of Djinn may establish non-inhabitation. A failed bounded higher-rank search does not establish it.

The Church harness requires the first two for every selected case. It never treats a timeout, an empty candidate list, or a search budget as a proof that the signature is uninhabited.

1.1 What practical completeness means here

The concrete acceptance obligation covers every signature in the Church source, including the local annotations. It also motivates general rules for arbitrary nesting, correlated instantiation, polymorphic arguments, and capture-safe substitution. The implementation does not identify cases by source function name or return a table of implementations.

No implementation can promise both termination and complete answers for all System F inhabitation requests: that decision problem is undecidable [2]. Curry-style System F type reconstruction and type checking are also undecidable [3]. This does not prevent independently checking an explicitly elaborated candidate, and it does not prevent useful bounded search. It does require precise reporting of the boundary between a checked success and an inconclusive search.

There are also ordinary language boundaries. GHC's `ImpredicativeTypes` extension enables Quick Look inference and permits quantification beneath type constructors and explicit polymorphic type applications. It does not make type-class instance arguments or implicit parameters impredicative [4]. Djex preserves the target language's restrictions instead of treating the extension as a license to manufacture arbitrary dictionaries.

2 Rank-N types and impredicative instantiation

2.1 Quantification inside a function type

Ordinary prenex polymorphism places type quantifiers at the outside:

```
identity :: forall a. a -> a
```

Higher-rank polymorphism permits quantifiers inside the type:

```
consumeIdentity :: ((forall a. a -> a) -> result) -> result
```

An inhabitant is

```
consumeIdentity consume = consume (\x -> x)
```

The callback expects one value that can be used at every type. Replacing its bound variable with an ordinary unification variable would weaken this requirement: a single use at one monotype would then be mistaken for a polymorphic value.

Introduction and elimination therefore require different operations. To construct a polymorphic value, the search opens its quantified variables as fresh rigid identities and constructs its body uniformly. To use an available polymorphic value, the search chooses an instantiation of its quantified variables. Each use may choose a different instantiation.

2.2 Selecting a polymorphic type as an argument

Predicative instantiation chooses a monotype for a quantified variable. Impredicative instantiation may choose a type containing a quantifier. With `Id = forall a.  a -> a`, the application of a provider of type `forall t.  t -> t` to an `Id` value selects $t := \texttt{Id}$. The selected type is a complete polymorphic type, not the body of a quantifier with its scope erased.

This distinction is visible beneath ordinary type constructors:

```
keepIdentities :: [forall a. a -> a] -> [forall b. b -> b]
keepIdentities xs = xs
```

The element schemes are alpha-equivalent. Their binder spellings differ, but each list element remains universally quantified. Comparing textual strings would be too strict; deleting the quantifiers would be unsound.

Quick Look is an influential example of using local structural information to choose impredicative instances without replacing an entire production type inference engine [1]. Djex uses a related engineering principle: the shapes already present in a query are valuable evidence for selecting instances. Its two search algorithms are not implementations of the Quick Look algorithm, and GHC remains a separate final checker for Haskell output.

3 Church encodings as a realistic corpus

The basic aliases are:

```
type Bool       = forall r. r -> r -> r
type Pair a b   = forall r. (a -> b -> r) -> r
type List a     = forall r. (a -> r -> r) -> r -> r
type Maybe a    = forall r. r -> (a -> r) -> r
type Either a b = forall r. (a -> r) -> (b -> r) -> r
```

The corpus adds `Tuple`, `Dict`, `Equal`, `LE`, and `Matrix`:

```
type Tuple a  = List a
type Dict k v = List (Pair k v)
type Equal a  = a -> a -> Bool
type LE a     = a -> a -> Bool
type Matrix a = List (List a)
```

The aliases hide substantial nesting. A dictionary contains universally quantified pairs as elements of another universally quantified encoding. A matrix contains polymorphic list values inside polymorphic lists. Expansion must preserve every binder and its dependencies.

The four local annotations are the `interleave` helper within `permutations`, `go` within `introsort`, `introselect` within `nthElement'`, and `go` within `setOp`. The manifest records the enclosing definition and source line for each. They are tested as generalized standalone signatures, without captured reference implementations.

3.1 Elimination requires a useful result type

Consider the Church pair projection:

```
first :: Pair a b -> a
first p = p (\x _ -> x)
```

The query requires choosing the pair's result type $r := a$. The selected continuation then has type $a \to b \to a$. The type is not inhabited merely by forwarding the opaque pair value; search must eliminate its quantifier.

A more revealing example is Church either elimination:

```
eliminateEither :: (a -> c) -> (b -> c) -> Either a b -> c
eliminateEither leftCase rightCase value = value leftCase rightCase
```

Here the useful selection is $r := c$. It correlates both continuation argument types with the desired result. Matching a provider's result suffix against the required result supplies this information directly.

3.2 Introduction under an argument boundary

Currying a Church pair requires constructing a polymorphic argument:

```
curryPair :: (Pair a b -> c) -> a -> b -> c
curryPair consume x y = consume (\continuation -> continuation x y)
```

The supplied lambda must work at every result type chosen by `consume`. A search that supports only a top-level universal quantifier cannot discharge this nested introduction obligation.

3.3 Transporting and transforming polymorphic components

Both engines synthesize pair exchange and pair mapping. The following forms are alpha-renamed presentations of generated Exference candidates checked by the whole-corpus harness:

```
swapPair :: Pair a b -> Pair b a
swapPair p continuation = p (\x y -> continuation y x)

mapPair :: (a -> c) -> (b -> d) -> Pair a b -> Pair c d
mapPair f g p continuation =
  p (\x y -> continuation (f x) (g y))
```

The input and output result quantifiers are distinct lexical scopes. Their relationship is established by the selected instantiation at a use site, not by globally identifying similarly named binders.

4 The shared representation and its invariants

The parser-independent foundation represents variables, constructors, type applications, arrows, products, and explicit universal quantifiers. A quantifier retains its binder list, class context, and body. The representation supports alpha-stable keys, free-variable analysis, substitution, and synonym expansion. Both backends use this foundation when crossing the public Djex API.

There are three relevant identities in the unification and instantiation logic:

Flexible variables may be solved by the current search equation, subject to ownership and scope checks.

Ambient rigid variables represent fixed types in the current context, including types opened by a legitimate introduction step.

Fresh comparison binders exist only while comparing corresponding quantified types. They must never escape into a substitution for an older flexible variable.

A source spelling is a presentation hint; it is not sufficient evidence that two variables have the same identity.

4.1 Capture-avoiding substitution

Substitution must traverse both a quantifier's context and its body while respecting shadowing. If a replacement contains a name that would be captured by a nested binder, the nested binder is renamed first. In particular,

$$[a := b](\forall b.\ a \to b) \quad \text{must denote} \quad \forall c.\ b \to c,$$

with fresh c, rather than $\forall b.\ b \to b$.

The Church extractor independently applies this rule to aliases. It gives each quantifier a fresh ASCII binder before substituting its type parameters. The resulting expanded type is then checked against the original aliases by GHC, so an error in this test-side transformation cannot silently redefine the acceptance type.

4.2 Whole polytypes remain whole

Quantified types remain intact when they become substitution images. A metavariable may receive a complete type such as $\forall a.\ a \to a$. Free-variable traversal still reaches the free variables inside this type; "opaque" does not mean invisible to occurrence, substitution, or scope checks. Only the bound variables are protected by their lexical scope.

5 Djinn: structural demands become checked instances

Djinn's propositional core uses LJT proof search. The higher-rank extension retains quantified types as alpha-stable atoms when they must be transported opaquely, and opens eligible positive occurrences for introduction. Checked instantiation bridges then expose useful hypothesis instances to the propositional search.

5.1 One coherent view for transport and introduction

A query may need to forward several existing polymorphic values unchanged while constructing other polymorphic values in the same result. Treating every positive quantifier as opaque prevents the constructions; opening every one loses the exact types needed for forwarding. Enumerating subsets of positive sites introduces an artificial boundary at the next omitted combination.

The transport-directed view makes the decision at each site in one traversal. A positive quantified scheme remains opaque exactly when an alpha-equivalent source scheme occurs at a negative position in the query or an actual provider. Otherwise the positive occurrence is introduced with fresh rigid variables. For example:

```
type Seven r = forall a b c d e f g.
  (a,b,c,d,e,f,g) -> r
type Id = forall x. x -> x

mix :: Seven r1 -> Seven r2 -> (Seven r1, Seven r2, Id, Id)
mix p q = (p, q, \x -> x, \x -> x)
```

The supplied `Seven` values retain their complete schemes, while each identity result opens for construction. The same decision scales to sixteen or twenty-four mixed sites without enumerating the corresponding subset frontier.

Loaded consumers must use the same view and namespace as the query. The environment therefore compiles query-aware argument views for real named providers; these views do not introduce a new provider or assume a missing implementation. Core search builds one coherent structural sequent, plus a nominal version when relevant, and checks every resulting proof in the normal proof environment.

For queries with at most eleven positive sites, this plan runs first only when its goal formula or an active provider's *symbol–formula pair* is absent from the established cached frontiers. A target-only premise is not evidence of such novelty. For larger queries, exact novelty enumeration itself can be costly, so the coherent view may run early without that exhaustive comparison. Eleven is an acceleration threshold, not a rank or site-count limit. With no available quantified scheme the extra plan is omitted; when the bounded novelty comparison proves the view already cached, the established alternative streams remain in place. The richer directed-instantiation family still waits until the earlier families and transport view have produced no inhabitant or definitive negative evidence. These decisions affect scheduling and useful representations, not source arity or the meaning of proof checking.

5.2 Constructing an argument at an impredicative instance

Selecting a polytype is only the first half of many practical examples. A provider may require a fresh value of that polytype, rather than accept one already present in the context:

```
buildComposition ::
  (forall a. a -> f a) ->
  (forall a. g a -> h a) ->
  (forall a. h a -> k a) ->
  f (forall a. g a -> k a)
buildComposition pack gh hk = pack (\x -> hk (gh x))

buildNested :: (forall a. a -> f a) ->
  f (forall b. b -> f (forall c. b -> c -> b))
buildNested pack = pack (\x -> pack (\_ _ -> x))
```

In the first example, a new rigid type parameter belongs to the argument being constructed. Both transformation providers must be instantiated at that same parameter. In the second, the inner construction depends on the outer argument's parameter and value. Flattening their scopes together would admit escaping variables; isolating every inner scope from its parent would discard this valid implementation.

Djinn's construction family therefore translates instantiated provider bodies at the polarity of an available premise. Quantified argument obligations open with private rigid identities. Each root construction owns an isolated family that can specialize the original checked providers within that scope and discover further obligations exposed by those instances. Descendants retain their actual scope dependencies; their private variables do not become general candidates in the enclosing query. One family shares the operational allowance of 512 expansion attempts after selecting its root and 64 retained premises including that root. Initial root proposals use the ordinary bounded instance-family builder separately.

Exact schemes retained for real loaded providers are included alongside quantified hypotheses discovered in the query, including schemes exposed only after applying preceding ordinary arguments. For a polymorphic goal requiring such construction, a small context runs after the primary search opportunity and before broad historical loaded-instance guesses. It contains the actual named scheme premises and one complete checked construction family. This ordering prevents unsuccessful older guesses from using the entire choice budget before a useful construction can be tried. An already available exact opaque source for the whole goal keeps the established forwarding route ahead of this speculative construction. For a monomorphic goal, the deferred phase also tries exact forwarding first. No scheduling choice adds an unchecked source assumption or changes the authority of negative evidence.

Scope ancestry follows the free variables of the original scheme and its selected type arguments. An independent later quantified result can begin a new scope; a dependent nested construction retains its deepest owner and that owner's ancestors. The reuse check rejects returning to an ancestor-active construction with its fixed private identities. Independent sibling uses of the same provider remain available: constructing a pair of polymorphic identities does not require an artificial sharing restriction.

The ordinary proof checker runs before the additional scope and reuse checks, and rejected alternatives do not refund search fuel. Kind checking temporarily renames exactly the owned private identities to fresh legal source names, while retaining the original identities in formulas and evidence. A private name outside the explicit ownership set is rejected. These steps provide usable construction scopes without treating a skolem as an unrestricted inhabitance assumption.

5.3 The shape of an instantiation bridge

For an available scheme

$$S = \forall a_1 \ldots a_n.\ B,$$

and a checked selection $\theta = [a_i := U_i]_{i=1}^{n}$, an elimination bridge relates the scheme to its specialized body $B\theta$. The bridge retains the original source scheme and the complete argument vector. Translation, kind checking, proof checking, and generated-term association remain responsible for admitting the instance.

The new directed family is additive. It does not change the meaning of existing opaque transport or polarized introduction rules. It also does not turn an unproductive bounded rule into a negative proof.

5.4 Match useful shapes instead of guessing all tuples

Let $\mathcal{D}$ be the deduplicated collection of eligible demands from the elaborated query and its retained type atoms. A demand may contain ambient variables, arrows, constructors, or a complete quantified subtree. Every free variable in it must belong to the allowed current scope.

The directed matcher compares the entire body B and each function-result suffix of B with each demand in $\mathcal{D}$. Only the leading variables $a_1, \ldots, a_n$ are selectable in the resulting leading-argument vector. Ambient source and demand variables remain rigid. When a result spine exposes a later context-free universal group, its temporary binders may participate in matching the eventual result; their solutions are discarded from the current vector. Ordinary checked application and elimination must still reach and instantiate that later group. Repeated occurrences of a selectable variable must produce alpha-equivalent selections. Source and demand binders are first made lexically unique using legal reserved source names, so identically spelled nested binders cannot be mistaken for outer parameters and ordinary kind validation can still inspect the selected types.

For the either eliminator, the scheme body is

$$(a \to r) \to (b \to r) \to r.$$

The final result suffix r matches the demand c, producing the correlated selection $r := c$. This avoids spending the choice-point budget on irrelevant instances before reaching the useful continuation type.

For a wide provider, one traversal can solve twelve or more correlated variables. The directed family's eligibility has no fixed six-binder cutoff. That statement is specific to this family: historical tuple-enumeration and provider-evidence routes retain their separately documented limits.

5.5 Nested quantifiers and scope protection

When a pattern and demand both contain quantifiers, the matcher establishes a lexical correspondence between their binders and compares the associated contexts and bodies. The correspondence takes precedence over outer selectable names. A selected image may not contain a demand-side binder introduced only by this nested comparison.

Thus a pattern $\forall b.\ a \to b$ may match $\forall c.\ \texttt{Int} \to c$ with $a := \texttt{Int}$. It must not match $\forall c.\ c \to c$ by selecting $a := c$, because c is not available outside that comparison. Alpha-renaming and scope checking are separate duties.

5.6 Unobserved choices and operational bounds

A result shape need not mention every quantified parameter. The remaining parameters draw from the same checked vocabulary. Repeated, or diagonal, selections are visited before the full Cartesian tail, which prevents an unproductive first coordinate from starving a useful repeated choice. Fully determined matches are emitted first. Partially determined matches then take turns contributing vectors, so guessing parameters for one match cannot indefinitely suppress another match's useful first selection.

Search-plan scheduling also matters. After the structural and nominal plans, focused plans try individual checked instantiation axioms to accelerate the first inhabitant. They are skipped once an earlier plan has produced a candidate. Rich compound demand-derived axioms are kept separate from the historical correlated family and act as an inhabitation fallback only when the earlier search has no result. This preserves established alternative ordering and cutoff behavior for already inhabited goals and avoids opening a large new alternative stream after a satisfactory result has been found. It is a guarantee about finding checked inhabitants under the configured search policy, not an exhaustive enumeration of every typable program.

The current family builder retains at most 16 axioms per scheme and 64 per family, with a 512-attempt bound. These are operational limits, not language rank limits. The directed rule remains positive-only. A search that misses an inhabitant because of these limits must remain inconclusive.

The raw proposed-vector allowance is applied before alpha-equivalence deduplication. Otherwise a request could traverse an unbounded sequence of duplicate or rejected proposals while apparently retaining only a few instances. A scheme whose allowance is exhausted stops before forcing the remaining lazy proposal stream. Resource accounting therefore bounds the work used to discover candidates as well as the number eventually retained.

The main implementation locations are `djinn/src-core/Djinn/Internal/DirectedInstantiation.hs`, `djinn/src-core/Djinn/Internal/Instantiation.hs`, and the checked query integration in `djinn/src-core/Djinn/Core.hs`.

5.7 Exact argument vectors supplied by a frontend

A frontend may already know the complete correlated type arguments of a retained provider. Such evidence is different from a heuristic request to guess arguments from a scalar vocabulary. Both backend adapters accept exact vectors through the shared provider-instantiation APIs, in forms with inferred argument kinds or explicitly supplied ground kinds.

The vector's required length is now derived from the validated retained provider's leading binder count n. Validation observes at most $n + 1$ cells of the caller's argument-list spine before traversing any argument contents. A short vector fails; an extra cell proves that a long or cyclic vector has the wrong arity. Thus a twelve-binder provider can receive twelve arguments without making infinite caller lists a termination hazard.

The exact-evidence route still limits a request to 32 assignment vectors, and each explicitly supplied kind has a 129-node structural preflight bound. Provider identity, exact argument count, substitution consistency, kinds, complete specialized type, and candidate association all remain checked. The historical six-argument constant continues to bound heuristic Cartesian enumeration; it is no longer a language bound on exact supplied vectors.

The facade regression suite exercises eight- and twelve-argument vectors for both engines, through both inferred-kind and explicitly kinded APIs, including GHC replay of the emitted applications. It also checks oversized and cyclic vectors. These tests distinguish a wider legitimate provider from

malformed external evidence, instead of simply increasing a global magic number.

Exact frontend assignments have dedicated priority plans as well as combined instance-family contexts. The historical families retain their relative order, while the coherent transport and loaded-construction accelerators may interpose when no earlier candidate has succeeded. Priority is an execution policy; every route retains the same evidence and target-language checking requirements.

6 Exference: structural equations between quantified types

Exference already introduces nested universal goals with branch-local rigid variables and instantiates scoped providers at independent use sites. The new unifier extends equations between quantified types while retaining whole polytypes as possible substitution images.

6.1 A motivating equation

Suppose a candidate must reconcile

$$\forall a.\ a \to \beta \quad \text{and} \quad \forall b.\ b \to \texttt{Int}.$$

Pure alpha-equivalence is insufficient: the free flexible variable β still needs to be solved. The correct solution is $\beta := \texttt{Int}$. Conversely,

$$\forall a.\ a \to \beta \quad \text{and} \quad \forall b.\ b \to b$$

must not solve the older β with the newly opened b.

6.2 Open corresponding binders in a private namespace

The unifier maintains a tagged representation distinguishing flexible variables, ambient rigid constants, and private bound-comparison identities. For an equation between two quantified schemes, it opens one corresponding binder from each side with the same fresh private rigid identity. Repeating this operation handles both grouped and successively written quantifier prefixes.

The remaining contexts and bodies become structural equations. Contexts are compared for the same class, arity, and argument structure. This operation does not solve a class constraint or create a dictionary; those obligations belong to the existing scoped constraint solver. Grouping and successive opening are supported for context-free prefixes and corresponding context layers. Moving a constraint across a quantifier boundary or treating differently ordered constraints as interchangeable is not part of this structural equation rule.

6.3 Occurrence and escape checks traverse polytypes

Before a variable is bound, the unifier checks:

1. the variable does not occur free in its own proposed replacement;
2. no private comparison binder occurs in that replacement;
3. substitution into outstanding equations and existing bindings remains capture-avoiding and structurally valid.

The same checks apply when a variable occurs inside a quantified substitution image. Updating all equations and already recorded substitutions prevents an indirect chain of bindings from hiding an escape.

The first condition rejects a cyclic solution such as $\beta := \forall a.\ a \to \beta$. The second rejects $\beta := \forall c.\ b \to c$ when b exists only inside the current quantified comparison. A complete closed image such as $\beta := \forall c.\ c \to c$ remains admissible.

The implementation is in `exference/src-core/Language/Haskell/Exference/Core/Unify.hs`. Candidate reconstruction and expression checking remain independent consumers of the selected substitutions. A unification success is not itself a target-language acceptance result.

6.4 Forwarding a whole scheme to a flexible argument goal

Structural unification is useful only when search presents the right equation to it. A provider may expect an argument whose type is still flexible. If a polymorphic value is available, eagerly eliminating all of its quantifiers loses the opportunity to choose the entire scheme as that argument's type.

Exference therefore retains a sibling search branch that forwards the available complete polytype to the flexible argument goal. The ordinary instantiated-use branch remains available. A representative shape is:

```
apply :: forall a r. a -> (a -> r) -> r
identity :: forall b. b -> b
consumeIdentity :: (forall b. b -> b) -> result

-- The argument metavariable of apply can select the complete Id scheme.
use :: result
use = apply identity consumeIdentity
```

The provider declarations in this example describe a contextual synthesis environment. The search result need not instantiate `identity` to a monotype before selecting it for `apply`. Its complete scheme supplies the correlated choice, and the ordinary expression checker validates the result. This change is in the Exference search module `Exference/Core/Internal/Exference.hs`; it does not weaken the checker.

6.5 A polymorphic provider exposed by a term application

Quantifiers can occur after ordinary value arguments, so useful provider schemes need not be visible at the beginning of an expression. Consider:

```
interleaved :: forall seed.
  seed -> (seed -> (forall a. a -> F a)) -> F (forall b. b -> b)
interleaved seed maker =
  let localFactory = maker seed
  in localFactory (\x -> x)
```

The first application exposes the scheme `forall a.  a -> F a`. Its next use must select the identity polytype and construct a polymorphic identity argument. The seed may have any type; neither a distinguished unit value nor a special factory name is required. GHC independently accepts this implementation at the displayed signature.

Exference's general partial-application rule already creates a local binding for the residual type and continues search in the extended lexical scope. The independent checker must preserve that same information boundary. A bottom-up inference pass over the let body could infer the identity

lambda as a monotype before discovering that the expected result is `F (forall b.  b -> b)`. Let checking therefore checks the binding at its retained annotation and checks the body at the trusted outer expected type. The annotation is independently validated; it is not an unchecked instruction to accept the desired result. Regressions exercise one and two ordinary arguments, nominal and ambient seed types, and rejection of an unrelated rigid annotation.

6.6 Successive term and quantifier layers

The same issue can repeat along one provider spine:

```
type Id = forall c. c -> c

alternating :: forall seed.
  seed ->
  (seed -> forall a. a -> seed -> forall b. b -> G a b) ->
  G Id Id
alternating seed maker =
  let secondFactory = maker seed @Id (\x -> x) seed
  in secondFactory (\x -> x)
```

The current instance of `a` must be selected before a later result quantifier exposes the final `G a b`. Exference uses a positive lookahead branch: it temporarily opens successive residual forall layers, peels their value arrows, and matches the ultimate result against the current demand. Only substitutions for the provider's current leading binders are retained. A retained image containing a fresh variable belonging solely to a future layer is rejected. Every value argument and every later provider use must still be synthesized and independently checked in its actual scope. Lookahead is an instance-selection heuristic, never a proof of the goal.

Source reconstruction must also preserve the instantiation boundary. The fully inlined term `maker seed (\x -> x) seed (\y -> y)` is accepted by GHC at this signature. In contrast, introducing two inferred let bindings without a type application can infer the first identity as a monotype and reject the final result. Applying `@Id` to an inferred let alias is also invalid, because that inferred binder is not a specified type-application slot.

The generated form above keeps the selected type application on the original applied-expression spine. Nonrecursive aliases used under explicit type applications are selectively inlined with the shared capture-avoiding substitution; other lets remain useful expected-type boundaries. The normalized expression is checked independently, as is any later repaired expression after the same normalization. No extra source declaration or unchecked type assertion is introduced. Ambient images use the scoped type patterns described below and receive the same final GHC check. Selective inlining can change sharing and runtime cost; type correctness does not certify an optimal evaluation strategy.

7 Haskell output and independent compiler checking

The Haskell acceptance pipeline is:

$$\text{type and context} \longrightarrow \text{checked request} \longrightarrow \text{candidate} \longrightarrow \text{rendered source} \longrightarrow \text{GHC acceptance.}$$

The library independently checks its internal candidate evidence; an ordinary public API call does not itself launch GHC. The corpus harness or the library's consumer supplies the external compiler replay shown in the final step. Each arrow has a distinct failure mode. A representable internal

type may require an explicit annotation in source. A correct quantified instance can be rendered incorrectly if its binder scope is lost. GHC's inference of an unannotated expression may choose a different instance from the one intended by search. The final compiler check detects these failures.

7.1 Nested instantiation can require explicit type evidence

Quick Look can infer many impredicative applications from their argument and result shapes, but that inference is not an unrestricted structural unifier beneath arbitrary universal quantifiers. Consider an abstract type constructor `F :: Type -> Type` and the query:

```
nested :: (forall a. F (forall b. b -> a))
       -> F (forall c. c -> (forall d. d -> d))
```

The correct choice is $a := \forall d.\ d \to d$. The source `nested p = p` nevertheless fails GHC's inference at this type. Explicit evidence records the successful choice:

```
nested p = p @(forall d. d -> d)
```

This example separates successful internal instantiation from successful source reconstruction. Returning the same unannotated variable after proving that it can be instantiated is insufficient.

The chosen polytype can also mention an ambient type variable:

```
nestedAmbient :: forall x.
  (forall a. F (forall b. b -> a))
  -> F (forall c. c -> (forall d. x -> d -> d))

nestedAmbient p = p @(forall d. _ -> d -> d)
```

The anonymous placeholder preserves the quantified shape while allowing GHC to recover the ambient `x` from the expected result. It avoids assuming that an internal fresh variable has the same textual name as the caller's scoped type variable. GHC 9.12.4 accepts these interior placeholders in visible type applications without requiring `PartialTypeSignatures`. They are inference variables checked against the original signature, not unchecked type casts or values supplied to synthesis.

The distinction between locally bound and ambient variables remains crucial. The explicit `d` above belongs to the polytype and must retain its binding relationship. Repeated ambient variables, two correlated ambient variables, and higher-kinded ambient variables such as `f` in `forall d. f d -> d -> f d` exercise this obligation further. An application such as `@(forall d. _ d -> d -> _ d)` still requires GHC to reconcile every placeholder with the same requested result type. The target compiler is the final authority on whether that reconstruction succeeds.

The shared generated-expression representation separates a fully inferred argument, a specified closed type, and a partially specified type pattern. The pattern preserves alpha-normalized lexical identities for all inner binders and uses anonymous holes only for ambient variable occurrences. The exact correlated instantiation remains part of the checked evidence; the pattern is source syntax derived from that evidence. The Exference expression checker reconstructs fresh metavariables for the holes and checks the whole application against its actual expected type. A pattern containing holes is not by itself a certificate that any instantiation is legal.

The required choice may become known only when an argument is supplied:

```
delayed :: (forall a. F (forall b. b -> a) -> result)
        -> F (forall c. c -> (forall d. d -> d))
        -> result
delayed p q = p @(forall d. d -> d) q
```

Here the result type alone does not determine `a`; checking `q` does. The unannotated source `p q` fails the same GHC inference boundary. Reconstruction therefore has to inspect the fully solved use, including its arguments, rather than only the substitution discovered while matching the provider's result. This obligation applies equally to scoped function arguments and named global providers, including when the selected polytype contains ambient variables.

Exference handles that timing by recording every ordinary local and global occurrence during the successful checker traversal, independently of optional term-graph construction. Abandoned checker alternatives roll back their traces. After solving the complete expression, the final substitutions are applied to the retained occurrence types and the trace is aligned with the original expression. A source-order mismatch rejects the candidate. Existing explicit global application spines retain their evidence instead of being decorated twice.

Only the necessary visible prefix is inserted: it ends at a selected binder whose polymorphic image cannot be inferred through its exclusively nested quantified occurrences. Earlier positions can use `@_` to preserve the source binder order. The entire rewritten expression is independently checked again before publication. This makes the completed-use repair an elaboration step supported by fresh checking, rather than a textual substitution applied to already printed code.

7.2 The source inventory is independent of synthesis

`test-church/extract_corpus.py` enumerates source signatures and aliases, then invokes GHC with:

```
ghc -v0 -fno-code -fno-write-interface -ddump-types \
    -fprint-explicit-foralls -dppr-cols=10000 \
    docs/examples/Church.hs
```

The original source typechecks under GHC 9.12.4. Its bodies are used only to resolve partial signature holes. For example, GHC resolves the omitted function type in `uncurry`, and the resulting full signature is the query. The extractor does not export a source expression to either backend.

The manifest records the source SHA-256, compiler version, source line, original signature, full resolved type, expanded AST, classification, and accepted assumptions. Its companion TSV is a compact projection consumed by the Haskell runner. The runner first regenerates the expected inventory in memory and rejects stale checked-in artifacts. The source hash is computed from UTF-8 text without a BOM and with line endings normalized to LF, so Windows and Unix Git checkouts have the same corpus identity.

7.3 Checking both sides of alias expansion

For each candidate, the generated compiler module contains two declarations:

```
candidate :: FullyExpandedQuery
candidate = generatedExpression

candidate_original :: OriginalAliasPreservingType
candidate_original = candidate
```

The first checks the exact expanded query. The second asks GHC to reconcile that query with the original aliases. For partial cases both declarations contain the same explicit default argument. The generated module enables `RankNTypes`, `ImpredicativeTypes`, `ScopedTypeVariables`, and `TypeApplications`.

For each inherently partial Haskell case the compiler fixture also checks:

```
candidate_partial :: ExactOriginalSignatureWithoutDefault
candidate_partial = candidate undefined
```

The synthesized helper's first argument is exactly the declared element default. The wrapper supplies bottom at that one boundary after synthesis; it does not expose a universal bottom provider to any search.

The module uses `NoImplicitPrelude`. Its ten Church aliases are copied as type declarations only. The search environment has an abstract `Int` type, and the 16 classified concrete-provider queries additionally have `churchZero :: Int`. No other named implementation, bottom value, or recursion helper is a candidate provider. The compiler module imports `undefined` solely for the explicit partial wrappers.

8 Partial signatures and inherently partial functions

Two different uses of the word "partial" must be kept separate. A *partial type signature* contains a source hole such as `_`; GHC can resolve that hole from the program. An *inherently partial function* has an original type with no total closed inhabitant in the intended parametric setting. Resolving a signature hole does not supply a missing element value.

Proposition 1. *There is no total closed implementation of* `forall a. List a -> a` *under the pure Church interpretation.*

Proof. Choose an empty element type E. The Church empty list $\lambda c.\lambda z.z$ inhabits `List` E. Applying the purported total implementation at E to this list would produce an element of E, which is impossible. □

The same argument applies to `fromJust` by supplying Church nothing, to `fromLeft` and `fromRight` by selecting the other alternative, and to dictionary lookup by supplying an empty dictionary. Extremum and seedless-fold signatures still admit empty input containers; the signatures do not express their informal nonempty precondition.

The explicit policy is to expose a default of exactly the required element type:

```
headWithDefault :: forall a. a -> List a -> a
fromJustWithDefault :: forall a. a -> Maybe a -> a
minimumWithDefault :: forall a. a -> LE a -> List a -> a
```

This is a contextual total implementation. It makes no claim that the original default-free signature has become total. Supplying an unrestricted `forall a. a` value would make every type trivially inhabitable and would invalidate the intended acceptance test; the harness does not do so. Haskell's exact original type is nevertheless checked by wrapping the synthesized helper with an explicit partial value at that single default argument. Lean keeps the stated default premise and introduces no bottom.

The 19 cases are `head`, `last`, `at`, `foldl1`, `foldr1`, `fromJust`, `fromLeft`, `fromRight`, `maximumBy`, `maximumOn`, `minimumBy`, `minimumOn`, `minMaxBy`, `minmaxElement`, `atKey`, `reduce`, `maximum`, `minimum`, and `minMax`. The manifest retains the original signature alongside the augmented query for every one.

9 Lean integration and its universe boundary

Leant reifies supported Lean types to the shared synthesis fragment, invokes the Haskell search backends, renders candidate Lean terms, and checks them against the original Lean goal. The result must pass Lean elaboration and kernel checking. Reification and Haskell search are not trusted replacements for this final check.

9.1 Lean Type is predicative

Haskell's impredicative type universe cannot be copied literally into Lean's `Type` hierarchy. Lean has `Type u : Type (u + 1)`, its universes are not cumulative, and universe-polymorphic declarations are instantiated at specific levels. `Prop` has its own impredicative behavior; it is not a substitute for arbitrary computational data [5].

For example, if a : `Type` u, a Church list that eliminates into r : `Type` v has the form

$$\Pi(r : \texttt{Type}\ v).\ (a \to r \to r) \to r \to r.$$

Its universe is `Type (max u (v + 1))`. Nesting such a list as an element of another encoding therefore imposes real universe constraints. It is incorrect to force every quantified type variable to `Type 0` and then assume every Haskell impredicative instance remains valid.

9.2 Universe-correct counterparts of the corpus

The Lean corpus generator consumes the expanded AST and translates every quantified type variable to an explicit binder of type `Type _`. Arrows and applications retain their structure, and Haskell `Int` maps to Lean `Int`. Each universe metavariable is constrained by elaboration of the goal and generated term.

This checks a valid universe instantiation of each Haskell signature. It does not claim that one implementation works for all independent assignments of universe levels, nor that Lean's computational universes are impredicative. The 19 default-bearing cases insert exactly the same element-default premise as the Haskell harness.

The frontend has explicit wire-format resource limits. Quantifier metadata and class-context argument lists each have their own 128-entry bound, while individual kind shapes retain the 129-node bound. Class argument width is independent of the old six-choice heuristic. The producer derives an exact provider vector's arity from its source binders, under its existing serializer depth fuel of 80, and the receiving backend validates that vector against the retained provider. These transport limits must be reported as such; they are not grounds for a mathematical non-inhabitation claim.

One small counterpart is:

```
example : (forall (a : Type _) (b : Type _),
    (forall r : Type _, (a -> b -> r) -> r) -> a) :=
  fun a b p => p a (fun x _ => x)
```

An abstract provider can also select a polymorphic value at a suitable higher universe:

```
example : (forall F : Type _ -> Type _,
    (forall a : Type _, F a) ->
      F (forall b : Type _, b -> b)) :=
  fun F supply => supply _
```

These examples are included in `Leant/test-church/UniverseProbe.lean`.

9.3 Replay and the prohibition on invented proof evidence

The Lean acceptance runner disables the general library provider inventory and classical fallback for pure and default-bearing cases. Integer-provider cases enable the explicitly defined zero and the ordinary available integer constructors. It records the exact accepted term for every query, writes all terms into standalone Lean declarations, and invokes Lean again on that file. The replay also inspects reported axioms for `sorryAx`.

An emitted term must therefore survive both interactive verification and standalone kernel replay. A declaration containing `sorry`, a forged axiom, or a missing query is a failure. A successful replay of 349 candidates does not compensate for a missing 350th candidate.

9.4 Implicit type binders are lambda-group boundaries

Lean output has a subtle additional scope obligation. Adjacent value lambdas can often be printed in one group, but an intermediate implicit universal quantifier can prevent that regrouping. Consider a callback that accepts a polymorphic identity and returns another polymorphic identity:

```
forall R : Type,
  (((forall {A : Type}, A -> A) ->
      (forall {B : Type}, B -> B)) -> R) -> R
```

The following candidate preserves the intermediate quantifier boundary:

```
fun _ f => f (fun _ => fun x => x)
```

Flattening its inner groups to `fun _ x => x` can make Lean assign the binders at the wrong expected-type boundary. Leant's renderer reconstructs lambda groups from the reified type fragment and starts a fresh group at each intermediate implicit quantifier. Initial implicit-binder elision retains its existing behavior. This is a source-rendering correction, independently checked by the Lean examples and kernel replay, rather than a new logical search rule.

9.5 Expected types must reach nested arguments and let aliases

An unannotated value lambda does not reveal whether its input is an ordinary monotype or a value with its own universal quantifier. For example, an application of a provider of type `forall a. a -> F a` to an identity lambda can require a polymorphic argument because the expected application result is `F (forall b. b -> b)`. Leant matches that expected result against the provider result and propagates the resulting capture-aware type substitution back into every argument domain. Lambda reconstruction then sees the necessary explicit or implicit type binders before emitting the argument.

An explicit closed structural type argument also supplies instantiation evidence at its original source binder before the final result expectation becomes available. This matters when a provider is partially applied, selects one or two polymorphic types, and then returns another quantified function. The renderer uses `visibleTypeArgumentClosedType` for this evidence and preserves the remaining quantifier layers. A partially specified argument containing anonymous holes is still rendered as a pattern, but is not treated as exact closed evidence.

A plain let alias retains its exact type, including universal quantifiers exposed after the preceding term arguments. Its body is fitted against the enclosing expected result. Silently opening that retained quantifier too early would hide the very boundary at which a later application selects a polymorphic type. The same distinction is relevant when an argument result contains unresolved variables introduced by opening a universal quantifier: such a result supplies no reliable type

constraint for the caller. The caller's expected argument type must instead guide reconstruction. Actual ambient free variables remain rigid throughout this process.

When an applied let alias still lacks an exact fitting domain, the renderer also considers a capture-avoiding normal form that inlines single-use let bindings. It refits that expression from the original goal; it does not duplicate a shared computation or perform eta contraction. The established renderings remain the first complete prefix. Each normal form retains its own bound of three render styles times 32 assignment choices, for at most 192 candidates across the two forms. A successfully fitted alternative can also recover from a primary rendering failure after strict source admission.

Visible selections at an implicit quantifier reached after an ordinary argument require positional syntax at the original known head. For example, Lean accepts `@factory seed ...`; it does not accept the analogous `@(factory seed) ...`. The renderer follows the complete checked source spine from that head and exposes intervening implicit type and instance slots with explicit arguments or inference placeholders. Exact named-provider evidence retains its separate metadata path: if flattening the complete spine also exposes a leading retained vector, its original forall-domain and binder visibility metadata are still used. An inconsistent retained vector fails instead of falling back to reconstructed evidence.

An open visible type annotation spells its known universal binders explicitly, even when the target type writes corresponding binders implicitly. A fitting-only visibility overlay aligns those known structural positions with the printed annotation after expected-type resolution. It preserves inferred type identities and the result fragment, leaves wildcard subtrees unchanged, and does not override retained exact provider vectors.

A closed annotation can similarly use explicit forall syntax while the expected result uses implicit binders. Final-result matching therefore ignores only the explicit/implicit flag of corresponding universal binders. It still checks the complete structure, rigid variables, correlations, and scope, and preserves the original result and annotation metadata. Known-argument matching and retained provider-evidence matching keep their stricter visibility checks. This separates the information needed to infer an application result from the information needed to print its value arguments correctly.

There is one further inference boundary when an argument's source type variable is still unresolved and another universal group occurs later in the provider's result spine. Lean may elaborate that argument before it knows that the argument must itself be polymorphic. For an implicit polymorphic identity, the renderer then writes the implicit type lambda explicitly:

```
factory seed _ (fun {_} value => value) seed _ ...
```

This supplies the lambda's binder shape without guessing its type. The same operation follows nested implicit groups inside the argument. It is restricted to unresolved source domains before a later quantified layer; already known annotation domains keep their existing rendering. A private marker introduced after name uniquification prints the anonymous implicit binder and has no term-variable occurrences. It creates no new type identity or provider and does not add a rendering cohort. The eight constructive declarations in `Leant/test-church/MixedSpineSyntax.lean` replay with empty axiom inventories.

Constructor-pattern reconstruction uses the actual scrutinee type to determine its field types. It cannot safely infer those types from the order of source names or unrelated quantified variables. Together, these corrections preserve the information needed by Church continuations, nested optional values, and association-list operations such as `lookupAll`. Every correction is tested by replaying the exact rendered Lean term.

9.6 Search fallback and bounded backend communication

Some Church goals return a quantified continuation after ordinary inputs that the matching implementation may ignore. If the earlier Exference search lanes find no term, Leant also retries without multi-constructor pattern branching. This preserves an opportunity to construct the continuation directly. The fallback uses the same type checking and candidate verification boundaries; it introduces no new provider or logical assumption.

Even unrelated standard datatypes can compete with the required partial applications in a bounded queue. If all earlier lanes still miss, Leant retries Exference with the transitive subset of the standard inventory needed by the goal, exact type assignments, and every retained foreign declaration. The closure follows aliases, datatype fields, class constraints and methods, instance prerequisites and heads, and tuple identities, including unit. All declarations supplied from Lean are preserved unchanged. The original full variable-name table and provider ratings are reused, and the candidate's authority records the actual smaller checked session. This is a positive fallback after an unsuccessful full-context search; it does not add a provider or establish negative evidence from the smaller context.

The subprocess boundary must enforce its requested time limit as well. On Windows, an owned process job and a pipe reader have different capabilities from their POSIX counterparts. Leant preserves ownership of the child process job and obtains reply lines through an owned reader with a bounded queue, so a blocked pipe read cannot silently disable the request timeout. These changes concern execution reliability; they provide no type-theoretic evidence in place of elaboration and kernel replay.

9.7 Optional provider assignments have their own work budget

Discovering a provider's ordinary type and discovering additional exact instance-derived type arguments are separate operations. The latter is optional: recursive instance search for a declaration such as `Or.by_cases`, whose constraints involve `Decidable`, must not discard that provider or the surrounding inventory merely because useful exact assignment evidence was not found.

Assignment discovery retains its limits of 32 inspected seed heads and sixteen complete vectors per provider. It additionally shares one allowance of 128 instance-resolution attempts across every seed head and every recursive constraint branch for that provider. The counter lives outside rollbackable metavariable state, so restoring a failed branch does not restore spent work. Each accepted vector still contains exactly the source-derived arguments and passes the existing completeness, kind, context, and identity checks. The attempt allowance restricts discovery work, not type arity or vector shape.

The optional computation also receives at most 2,000 additional Lean heartbeats, clamped to the remaining outer command allowance. It never resets or enlarges that outer budget. Ordinary Lean errors and exhaustion of the local heartbeat allowance omit optional assignment evidence while preserving the provider and earlier and later inventory entries. Attempt-quota exhaustion can return the already completed vector prefix. Outer command exhaustion, interrupts, and other runtime exceptions still propagate. Debug diagnostics in the Haskell driver retain the actual Lean discovery error, so a failed discovery is distinguishable from an empty successful inventory.

Five standalone Lean regressions pass for this boundary: actual discovery through the recursive `Decidable` case, an ordinary error, local heartbeat exhaustion, recursion-error propagation, and interrupt propagation. These checks establish the behavior of the emitted Lean discovery code and supplement the rebuilt executable's independent corpus and fixture checks reported below.

10 Examples from simple to demanding

The examples below separate the structural capability exercised from any informal behavioral expectation.

10.1 Basic construction

```
pair :: a -> b -> Pair a b
pair x y continuation = continuation x y

nil :: List a
nil _ seed = seed
```

Both require introducing a result quantifier. The pair additionally captures two outer values without specializing the result type.

10.2 Nested continuation consumption

```
uncurryPair :: (a -> b -> c) -> Pair a b -> c
uncurryPair combine p = p combine

consumePolymorphic :: ((forall a. a -> a) -> r) -> r
consumePolymorphic consume = consume (\x -> x)
```

The first eliminates a supplied universal value at a chosen result type. The second introduces a universal value inside a callback argument.

10.3 A polymorphic payload inside an abstract constructor

```
-- A synthesis query; F is an abstract type constructor.
(forall a. f a) -> f (forall b. b -> b)
```

The provider must be instantiated at a complete polytype. Expanding or erasing that polytype would change the requested result. The corresponding Lean example above chooses consistent universe levels rather than assuming same-level self-containment.

10.4 Wide correlated instantiation

```
(forall a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 a11 a12.
   (a1,a2,a3,a4,a5,a6,a7,a8,a9,a10,a11,a12))
 -> (x12,x11,x10,x9,x8,x7,x6,x5,x4,x3,x2,x1)
```

Matching the result tuple solves twelve related selections together. This is a targeted regression for the removal of the historical fixed binder-count restriction from the directed matching family. The premise is intentionally very strong; this example tests provider instantiation, not the construction of a closed inhabitant of that premise.

10.5 Nested dictionaries and matrices

The corpus includes signatures such as:

```
collapse :: Equal k1 -> Equal k2 -> Dict k1 (Dict k2 v)
         -> Dict (Pair k1 k2) v

mapAccumL :: (s -> a -> Pair s b) -> s -> List a
          -> Pair s (List b)

transpose :: Matrix a -> Matrix a
```

The first has quantified encodings nested in dictionary keys and values; the second combines a state argument, a polymorphic pair result, and a polymorphic list result; the third moves through two layers of list encoding. The acceptance test permits any matching inhabitant. For example, returning an empty dictionary satisfies `collapse`'s type. A requirement to actually flatten all entries must be specified and verified separately.

11 Acceptance results and reproduction

The source inventory has canonical UTF-8/LF SHA-256

`782e4edaa5bf813e30e39ae02d52278ab0566315ebc521401a947b98c44cfd11`

and was extracted using GHC 9.12.4. The four local annotations are included in the total-case count.

Case class	Signatures	Djinn + GHC	Exference + GHC
Pure total context	315	315	315
Concrete integer provider	16	16	16
Explicit element default	19	19	19
All signatures	350	350	350

Every Haskell success above includes both the expanded query declaration and the alias-preserving forwarding declaration. The 19 partial cases per engine also include an exact-original-signature partial wrapper. Thus the run checks 700 synthesized helpers through 1,438 typed declarations: 700 expanded declarations, 700 alias-preserving forwarding checks, and 38 explicit partial wrappers. The forwarding declarations and wrappers are checks and adapters, not additional synthesized candidates. Every original signature is checked; the 19 original partial signatures remain explicitly partial.

The final explicit-budget Haskell replay uses Djex implementation revision

`e2eb71e321c523966e47e6eedb1e1d5c14f4e7de`

with 10,000 steps or choices and a two-second timeout per query. The runner rechecks the source inventory before synthesis. Its generated modules, per-query results, and compiler diagnostics are retained under `test-church/results/published-layered-rankn`; a local provenance record also identifies the tested executable by SHA-256.

From the Djex repository root:

```
python test-church/extract_corpus.py --check
cabal test djex-church-tests --test-show-details=direct \
  --test-options="--backend both --timeout 2 --steps 10000"
```

For a development run against an already built library:

```
cabal exec -- runghc -package=djex test-church/Main.hs \
  --backend both --timeout 2 --steps 10000 \
  --report-dir test-church/results/acceptance
```

The backslash continuations are typographical shell notation; on PowerShell, enter the command on one line or use its native continuation syntax. The runner defaults to both engines and records every query result, generated module, and full compiler diagnostic output.

The independent scope and reconstruction probe is run with:

```
cabal exec -- runghc -package=djex test-church/probe_scopes.hs
```

It sends 38 positive and 12 negative type queries through each engine, covering alpha-equivalence, shadowed binders, nested positive quantifiers, ambient-variable correlations, higher-kinded application patterns, fresh polymorphic argument construction, composition and dependent nesting, and successive quantified provider results exposed after ordinary term arguments. The negative cases include direct, indirect, nested, and higher-kinded scope escape attempts. An absent candidate is recorded only as bounded search evidence; a timeout is not accepted as a passing negative result. Positive queries receive 10,000 steps or choices, and every negative query receives the same smaller budget of 1,000. All retain a three-second wall-clock safety timeout. Every returned term is checked by GHC, even if its query was expected to have no candidate. Five cases provide named polymorphic functions through their signatures only; their total implementation bodies live in a separate compiler support module whose constructors are hidden. Every generated candidate module imports exactly its permitted provider names. One additional case supplies a unit value; the remaining cases receive no named value providers.

On the validated build, all 100 queries met their expected outcomes: 76 positive implementations passed GHC, and 24 negative queries returned no candidate under their search bounds, without errors or timeouts. These adversarial checks are additional evidence, separate from the 700 Church implementations and their alias/default wrappers.

The complete Djex validation covers all 19 registered test components: 1,930 ordinary test cases, separately from the 700 Church synthesis cases and 100 scope queries reported above. An aggregate run and a rerun of the repaired subset close all components successfully. This includes the existing API, frontend, proof-search, semantic Length, and command-line regression suites, so the corpus result is accompanied by checks of the established behavior.

Windows validation retains platform-appropriate absolute executable paths and the corresponding exact policy fingerprints. A short-deadline test also uses a fake worker that waits without later writes after recording its status wait. A delayed response could otherwise begin a trace write during process termination. The test keeps its 200-millisecond query deadline, 2,000-millisecond shared allowance, required event records, and strict trace parser. These portability and fixture corrections are separate from the rank-N rules.

From the Leant repository root after building the current executable, run each engine independently and retain separate reports:

```
python test-church/run_corpus.py --manifest ../Djex/test-church/manifest.json \
  --leant PATH_TO_LEANT_EXECUTABLE --engine djinn --window 1 \
  --steps 4096 --timeout 30 --output test-church/generated-djinn-acceptance
python test-church/run_corpus.py --manifest ../Djex/test-church/manifest.json \
  --leant PATH_TO_LEANT_EXECUTABLE --engine exference --window 1 \
  --steps 4096 --timeout 30 --output test-church/generated-exference-acceptance
```

The normal manifest path is `lib/Djex/test-church/manifest.json`; the explicit path above supports development with the separate Djex checkout. The Lean runner's JSON report and standalone replay output, rather than the Haskell counts, are the evidence for its target-language results. The separate `-engine both` option runs Leant's combined search mode; success in that mode does not establish that either individual engine succeeds alone.

The final independent live runs each produce 350 of 350 candidates, and Lean 4.32.0 accepts all 700 exact displayed terms in standalone replay. Every one of those declarations has an empty axiom inventory. As in the Haskell inventory, each engine covers 315 provider-free cases, 16 integer-provider cases, and 19 cases with an explicit element default. The last group remains conditional on that default in Lean, while Haskell separately checks its documented partial wrapper at the original signature.

Both runs use a one-candidate window, 4,096 search steps, and a thirty-second per-query timeout. They use Leant implementation revision `4757569`, with Djex revision `e2eb71e`, on the same unchanged executable with SHA-256

`addfac35b9d82955fc871c177b582a8c043475c0171c22cb17977e0e9f5b9869`

The live-synthesis reports are `Leant/test-church/generated-djinn-acceptance/results.json` and `Leant/test-church/generated-exference-acceptance/results.json`. They record source and manifest hashes, executable stability, every candidate, standalone kernel results, and the axiom inventories. This is fresh synthesis and exact-term replay for each backend, not a replay-only substitution for a missing engine run.

The compact fixtures run with:

```
python test-church/run_fixtures.py --leant PATH_TO_LEANT_EXECUTABLE
```

Their inventory contains nine representative Church queries, twenty-three broad rank-N targets repeated in each of the three engine modes, six exact wide-provider queries, and six named-provider queries with successive quantified result layers, for 90 queries altogether. The broad targets include genuine rank-seven continuation encodings with explicit and implicit quantifiers, composition and dependent construction, and successive quantified result layers, including ambient open choices and explicit/implicit source and target quantifier combinations. The eight- and twelve-binder provider tests measure argument width, which is distinct from type rank. The named layered factories additionally exercise discovery, source reconstruction, and repeated impredicative choices at global application sites. These twelve provider queries intentionally declare opaque primitive premises and require exactly that declared axiom inventory; the closed Church and broad rank-N fixtures require empty axiom inventories.

All 90 compact fixture queries pass on that same unchanged executable. The standalone Lean 4.32.0 replay accepts every exact displayed term. Of these, 78 declarations have empty axiom inventories, and twelve have exactly their documented supplied-premise inventories. The report is `Leant/test-church/generated-fixtures-bounded-final/results.json`.

The broader ordinary transcript inventory contains 30 files, 263 explicit-type queries, and two proof-mode `:synth` commands, for 265 synthesis commands in total. Those static counts describe

its coverage, separately from any particular replay result. The Python acceptance harness also checks the order of interleaved option changes and queries, so a later setting cannot silently alter an earlier query's test conditions; its 14 harness regression tests pass. The ordinary Leant Haskell unit suite additionally passes all 558 tests. These unit and harness checks are distinct from the standalone target-language replays of the compact fixtures and full corpus.

The full ordinary transcript replay on that same executable remains pending. That additional compatibility check is separate from the completed 700-case Lean corpus and 90 compact fixture checks above.

12 Limits, diagnostics, and future extensions

The changes remove concrete structural barriers; they do not remove the need for bounded search. The following limits remain explicit:

- The directed Djinn route handles context-free leading schemes. Existing constrained-provider and dictionary rules retain their own eligibility conditions. Structural context equality in Exference is not constraint solving.
- The directed matcher uses supplied query structure and a finite vocabulary for unconstrained parameters. It does not enumerate every possible intermediate polymorphic type.
- Historical Djinn instance families, retained-axiom counts, tuple attempts, Exference queue limits, and user time/step budgets can still make a search inconclusive.
- Optional Lean provider-assignment discovery has a shared per-provider attempt allowance and a local heartbeat budget. A provider may therefore remain available without every useful exact assignment being discovered; missing optional evidence is not a non-inhabitation result.
- General higher-rank subsumption, polymorphic-let inference, and arbitrary higher-order type unification are not claimed as complete algorithms. Scope-safe quantified structural equality is a narrower, auditable operation.
- Partial source signatures require an elaborated expected type. The corpus uses GHC to resolve them; a bare underscore without program context is not a complete synthesis specification.
- Lean validates universe-correct counterparts. The translation does not identify `Type u` with `Type (u + 1)`, assert cumulative universes, or replace computational types by propositions.
- Corpus success means matching type signatures under the documented assumptions. It says nothing by itself about runtime cost, termination of imported providers, or equivalence to the source algorithms.

A useful future extension should add a general rule, preserve the scope and evidence invariants, include a failing practical example, and replay emitted terms in the actual target language. Expanding a numeric cutoff alone is not a substitute for understanding which structural information a query already provides.

A Implementation and artifact map

Component	Responsibility
`Djinn/Internal/DirectedInstantiation.hs`	Structural demand matching, correlated leading-binder selection, nested scope correspondence, and unobserved-choice scheduling.
`Djinn/Internal/Instantiation.hs`	Scheme eligibility, vocabulary scope checks, bridge construction, deduplication, and operational family bounds.
`Djinn/Internal/TypeFormula.hs`	Coherent transport-versus-introduction views and
`Djinn/Internal/Environment.hs`	query-aware formulas for real loaded providers.
`Exference/Core/Unify.hs`	Structural quantified equations, private rigid binder identities, capture-safe polytype substitution, and occurrence/escape checks.
`Synthesis/Generated.hs`	Closed and partially specified visible type-argument syntax, lexical binder identities, and matching of emission patterns against selected types.
`Exference/Core/Internal/ExpressionCheck.hs`	Independent checking, completed occurrence evidence, late instantiation reconstruction, and rechecking of the rewritten expression.
`Exference/Core/Internal/Exference.hs` `Polytype.hs`	Result-directed selection across successive quantified result layers, with actual argument obligations retained by ordinary search.
`Exference/Core/Expression.hs`	Capture-avoiding normalization of let aliases used at visible type-application boundaries.
`test-church/extract_corpus.py`	Source inventory, GHC hole resolution, alias expansion, contextual defaults, and artifact freshness.
`test-church/manifest.json`	Full provenance, source and resolved signatures, expanded ASTs, scope, and case classification.
`test-church/Main.hs`	Public-API synthesis through both Haskell backends and independent GHC checking against expanded and alias-preserving types.
`test-church/probe_scopes.hs`	Independent alpha-renaming, scope, correlation, higher-kinded application, and nested-source-reconstruction queries with GHC replay.
Leant: `src/Leant/Synth/Render.hs` `src/Leant/Synth/Engine.hs`	Expected-type-guided Lean reconstruction, retained let schemes, constructor fields, lambda boundaries, and the checked search fallback.
Leant: `src/Leant/Synth/Fragment.hs`	Type reification, exact provider metadata, and separate transport resource bounds.
Leant: `src/Leant/Backend.hs`	Owned backend processes, bounded response reading, request deadlines, and cleanup.
Leant: `test-church/run_corpus.py`	Universe-aware Lean goals, explicit provider policy, exact candidate capture, and standalone kernel replay.
Leant: `test-church/UniverseProbe.lean`	Small universe and polymorphic-argument examples checked by Lean.
Leant: `test-church/run_fixtures.py`	Exact displayed-term replay for compact Church, broad rank-N, live eight- and twelve-argument providers, and named layered factories.

References

[1] Alejandro Serrano, Jurriaan Hage, Simon Peyton Jones, and Dimitrios Vytiniotis. *A Quick Look at Impredicativity*. Proceedings of the ACM on Programming Languages 4 (ICFP), article 89, 2020. doi:10.1145/3408971. Author-hosted paper.

[2] Andrej Dudenhefner and Jakob Rehof. *A Simpler Undecidability Proof for System F Inhabitation*. TYPES 2018, LIPIcs 130, 2:1–2:11, published 2019. doi:10.4230/LIPIcs.TYPES.2018.2.

[3] J. B. Wells. *Typability and Type Checking in System F Are Equivalent and Undecidable*. Annals of Pure and Applied Logic 98(1–3), 111–156, 1999. doi:10.1016/S0168-0072(98)00047-5. Earlier conference paper.

[4] GHC contributors. *GHC 9.12.4 User's Guide, section 6.4.21: Impredicative polymorphism*. Versioned GHC reference chapter.

[5] Lean contributors. *The Lean Language Reference: Universes*. Lean reference chapter.